bigt.ai

P2P Settlement — Wallet-Native CNY Swap

What this is. A peer-to-peer swap between crypto on **Bigtangle L0** and CNY fiat (WeChat Pay / Alipay / bank). The seller escrows tokens in a **2-of-3 P2SH address **(seller, buyer, engine); the buyer pays the seller** directly** in their own payment app; the seller confirms receipt on their own statement and the escrow releases. Every irreversible step is signed with the wallet's own PQ key, and the engine can only ever **co-sign** a spend — it never holds your funds and never touches fiat.

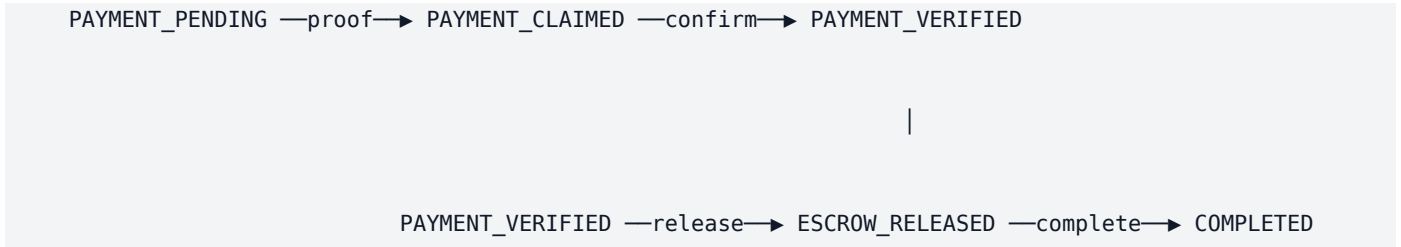
Where. The **P2P** screen in the wallet (sidebar → Trade → P2P). Everything below is captured from that screen on the local demo build.

Demo vs. live. The screenshots come from the demo build: the engine runs with a memory store and no chain check, so the lock/release legs use synthetic transaction hashes. The state machine, the signatures, the append-only event log and the on-chain audit anchor are the real thing; only the L0 broadcast is stubbed.

Seller confirmation is the fiat signal. There is no collection API for a personal WeChat/Alipay/bank account, so there is no machine-verifiable payment event to wait on. The seller checks **amount + remark + time + transaction number in their own app** — a screenshot never counts — and confirms. Design details live in `docs/p2p.md`.

The flow at a glance

```
ACTIVE ──match──▶ MATCHED ──lock──▶ ESCROW_LOCKED ──instructions──▶ PAYMENT_PENDING
                                                                          |
```



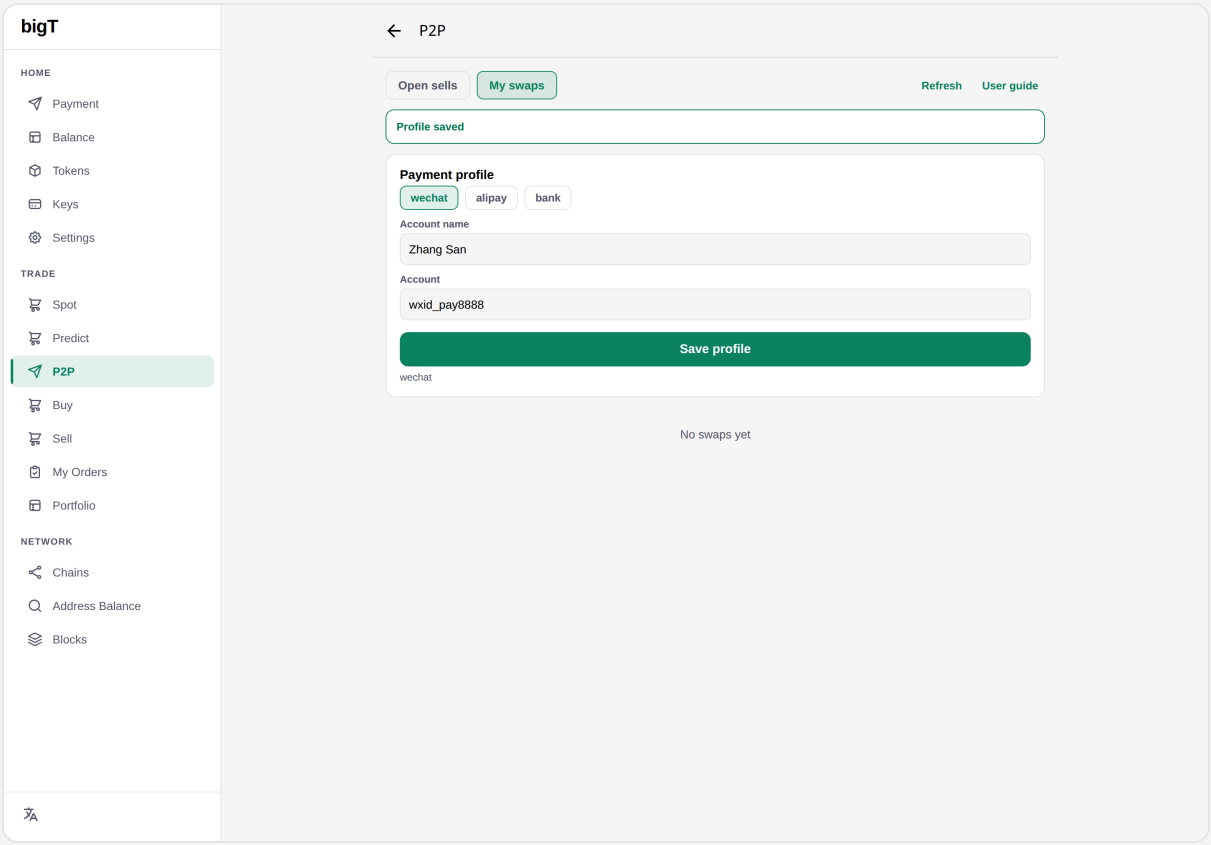
#	Step	Who acts	On-chain / engine
1	Seller saves a CNY collection profile	seller	profile stored server-side, never in the book
2	Seller lists a signed sell order	seller	order stored, not funded
3	Buyer matches (receive address)	buyer	`swapId` created, escrow address derived
4	Seller funds the 2-of-3 escrow, engine proves the lock	seller + engine	`ESCROW_LOCKED` on L0
5	Buyer pulls the payment instructions	buyer	`PAYMENT_PENDING` (+ per-swap remark)
6	Buyer pays in their own app, claims with the transaction number	buyer	`PAYMENT_CLAIMED`
7	Seller checks their own statement and confirms	seller	`PAYMENT_VERIFIED`
8	Engine releases the tokens, swap completes	engine	`ESCROW_RELEASED` → `COMPLETED`

Every transition is anchored as a `social.p2p-swap` record on the L1-SOCIAL chain, so the whole lifecycle is publicly auditable without exposing either party's account details.

---

1. The seller saves a collection profile

On the **My swaps** tab the seller picks a rail (WeChat, Alipay or bank) and enters the collection name and account. The profile is stored server-side under the seller's DID and is **not** part of the public order book — it is revealed only to the matched buyer by the instructions call.

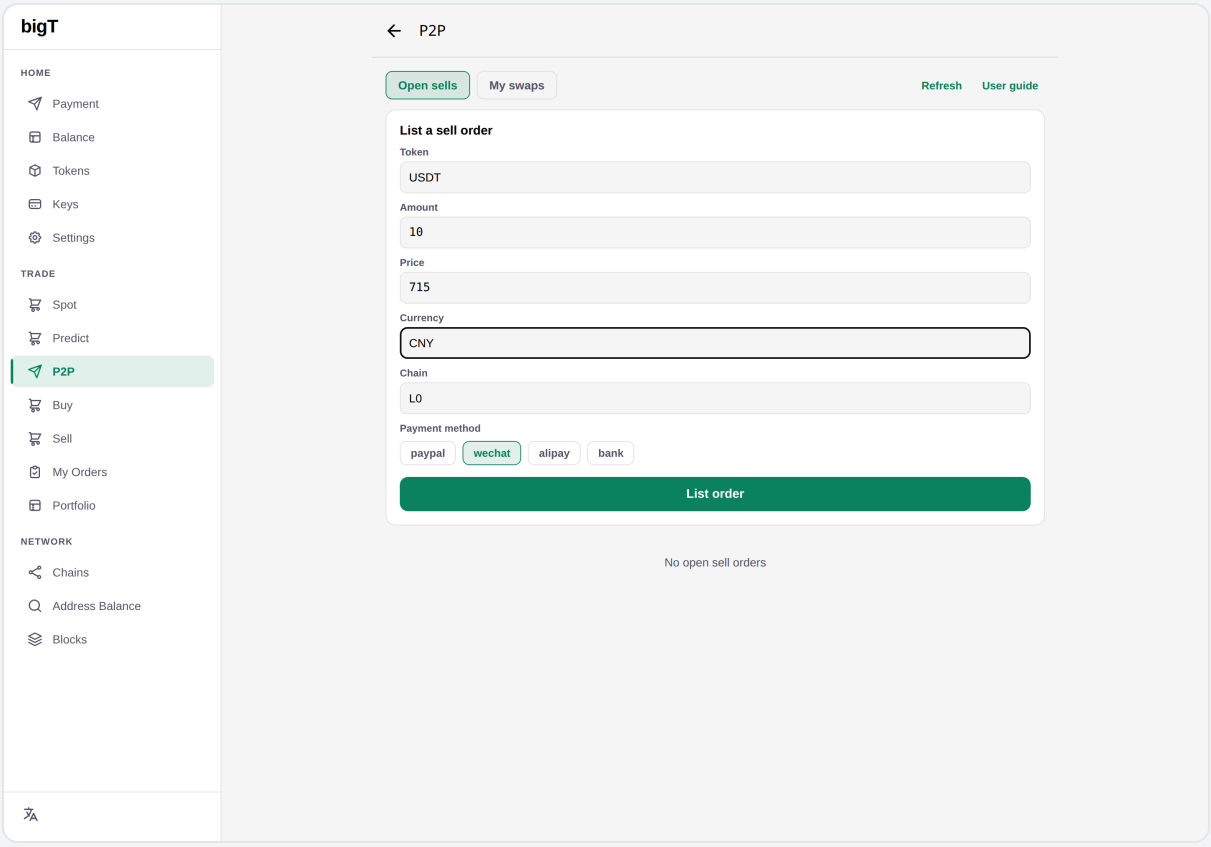


---

2. The seller lists a sell order

On the **Open sells** tab, fill in the order: the token and amount you give, the CNY price you want, the chain the tokens sit on, and the **payment method** (wechat / alipay / bank). Selecting a CNY rail switches the currency to CNY.

Terms are locked into the order when you list it, so this is signed with your wallet key:

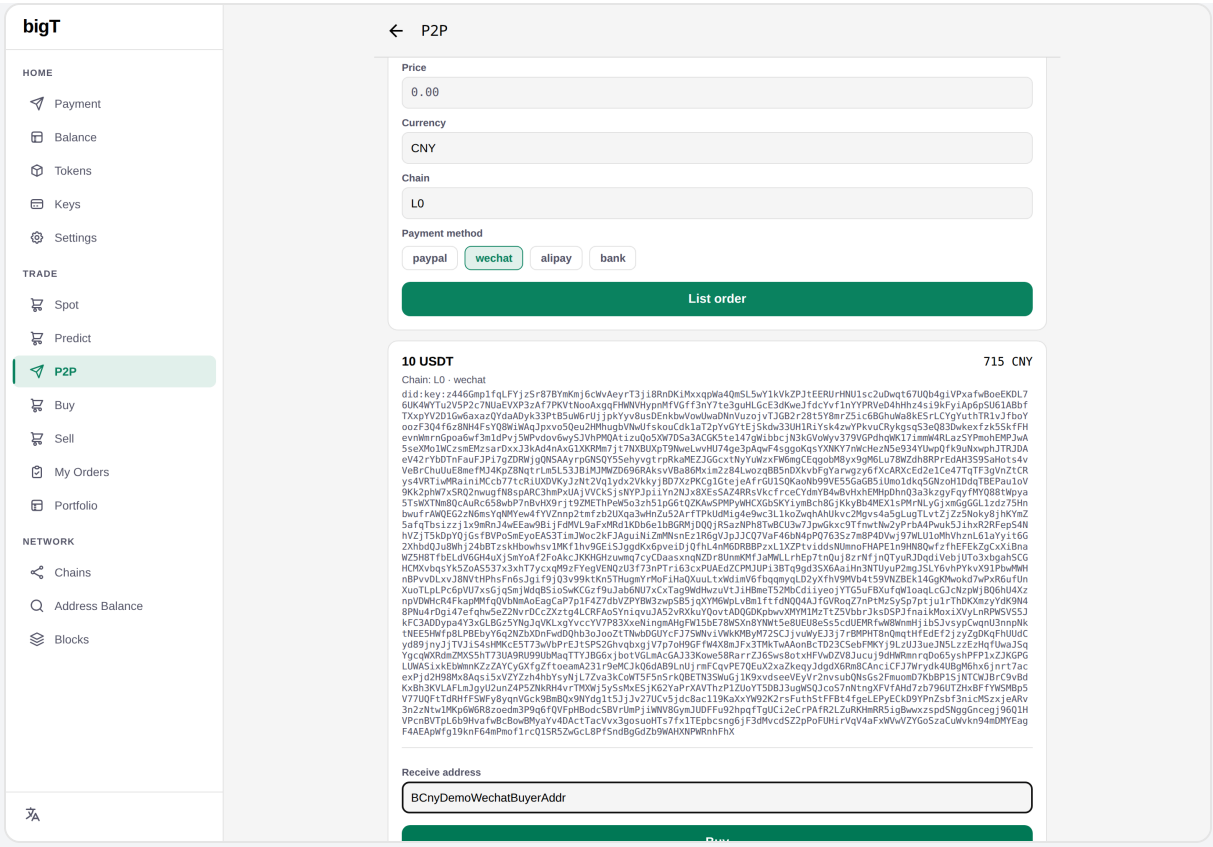


Once listed, the order is `ACTIVE` in the engine and appears in the public order book (it carries no personal details).

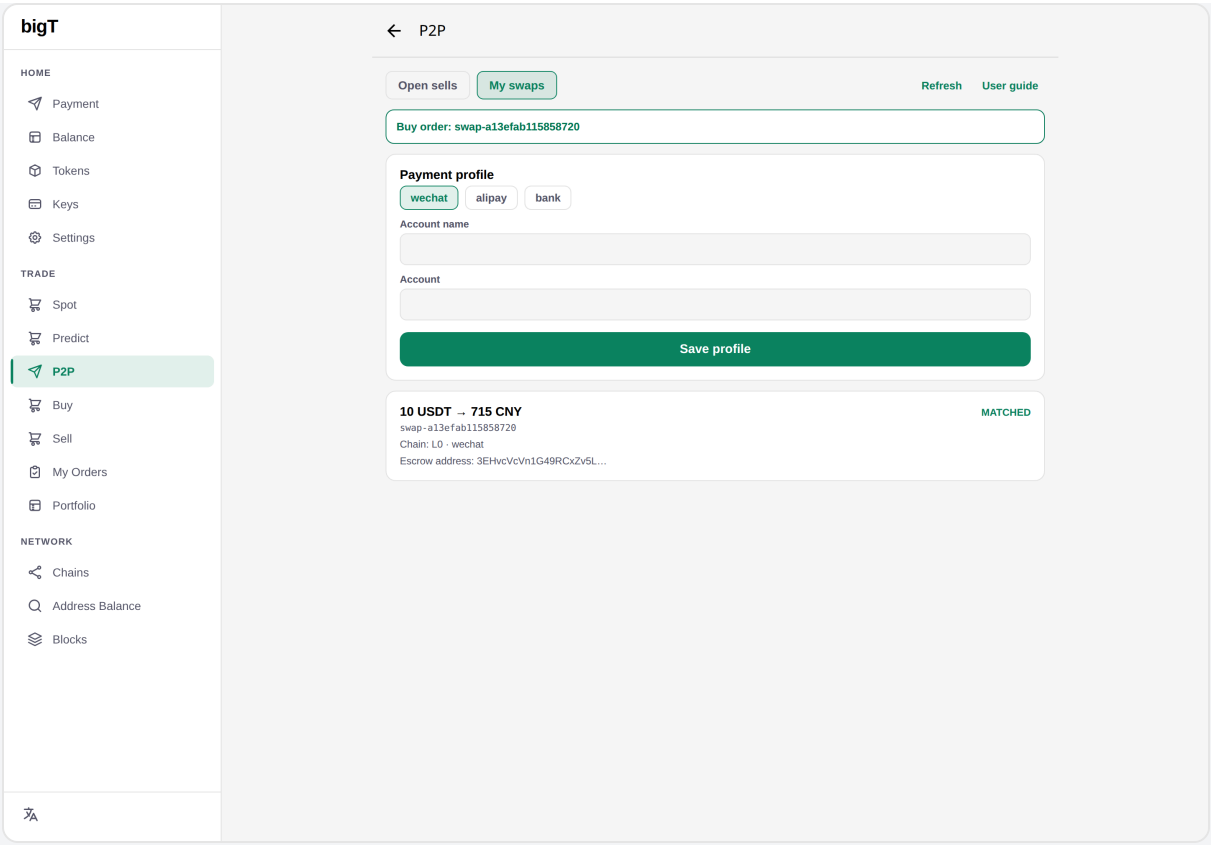
---

3. The buyer matches

The buyer opens the order and supplies the **receive address** for the released tokens. Matching commits the buyer to pay, so it is signed too:



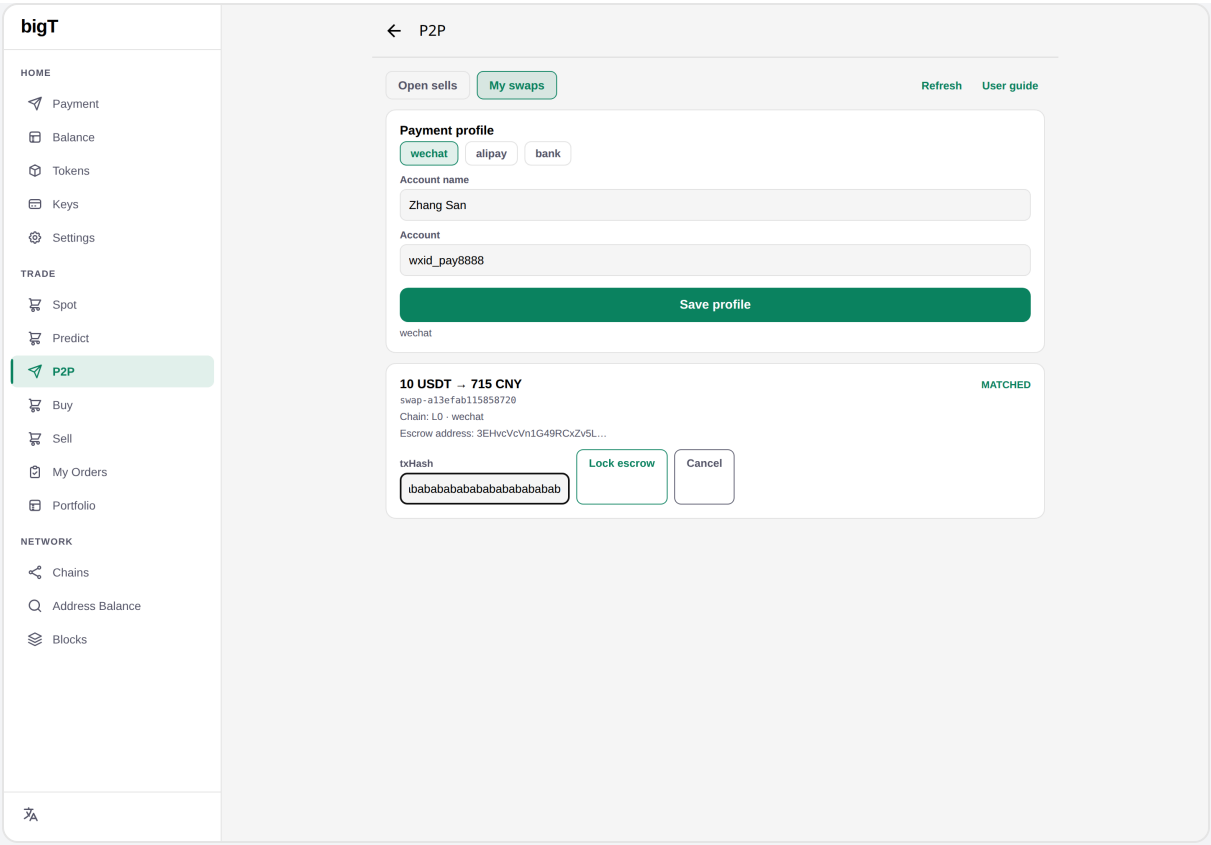
The engine creates a unique `swapId` for the lifecycle and the swap moves to `MATCHED`, shown on the buyer's **My swaps** tab:



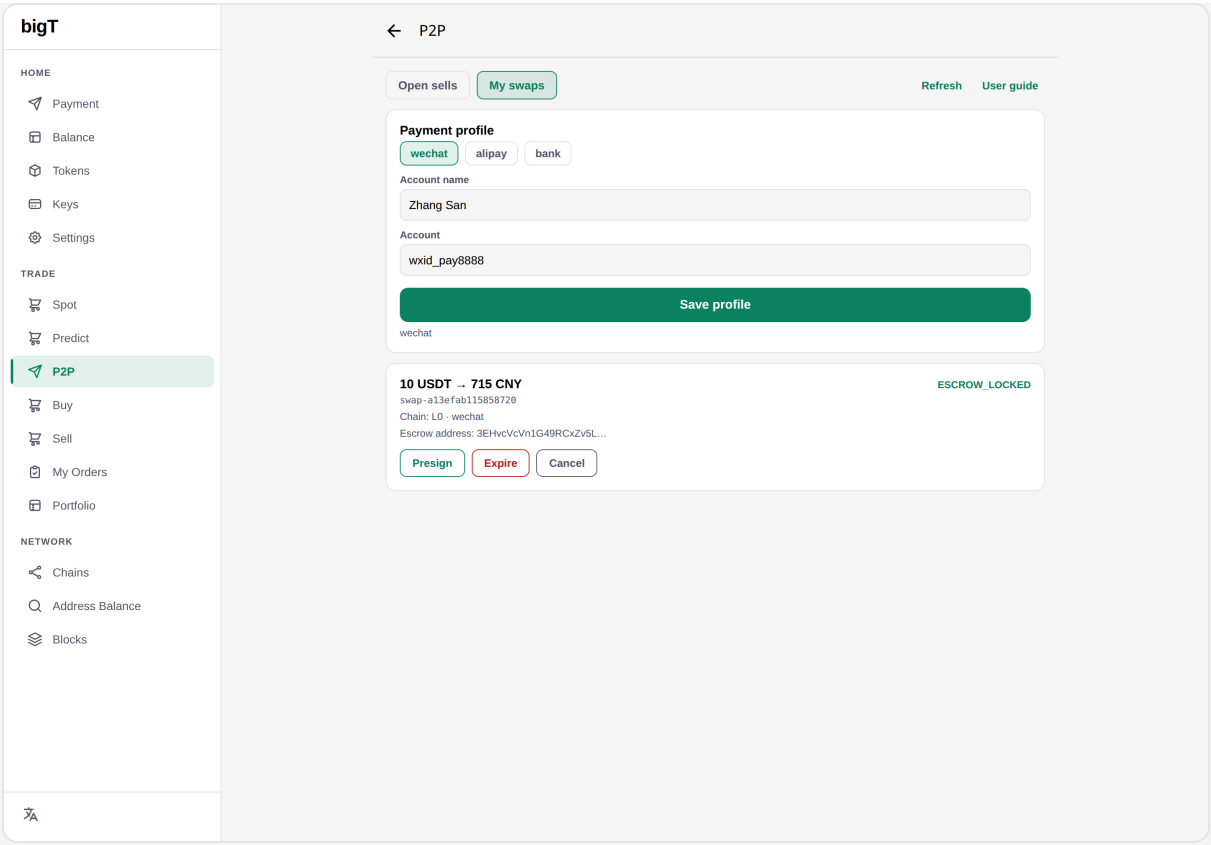
---

4. Escrow lock — the seller funds L0

The seller sends the tokens to the deterministic 2-of-3 escrow address and reports the transfer's `txHash`. The engine proves the lock from the chain (`CONFIRMED`, destination `escrowAddress`, amount) and fails closed if any check does not hold:



The swap is now `ESCROW_LOCKED`. The buyer sees the funds secured **before** paying any fiat, and gets a **Payment details** action:



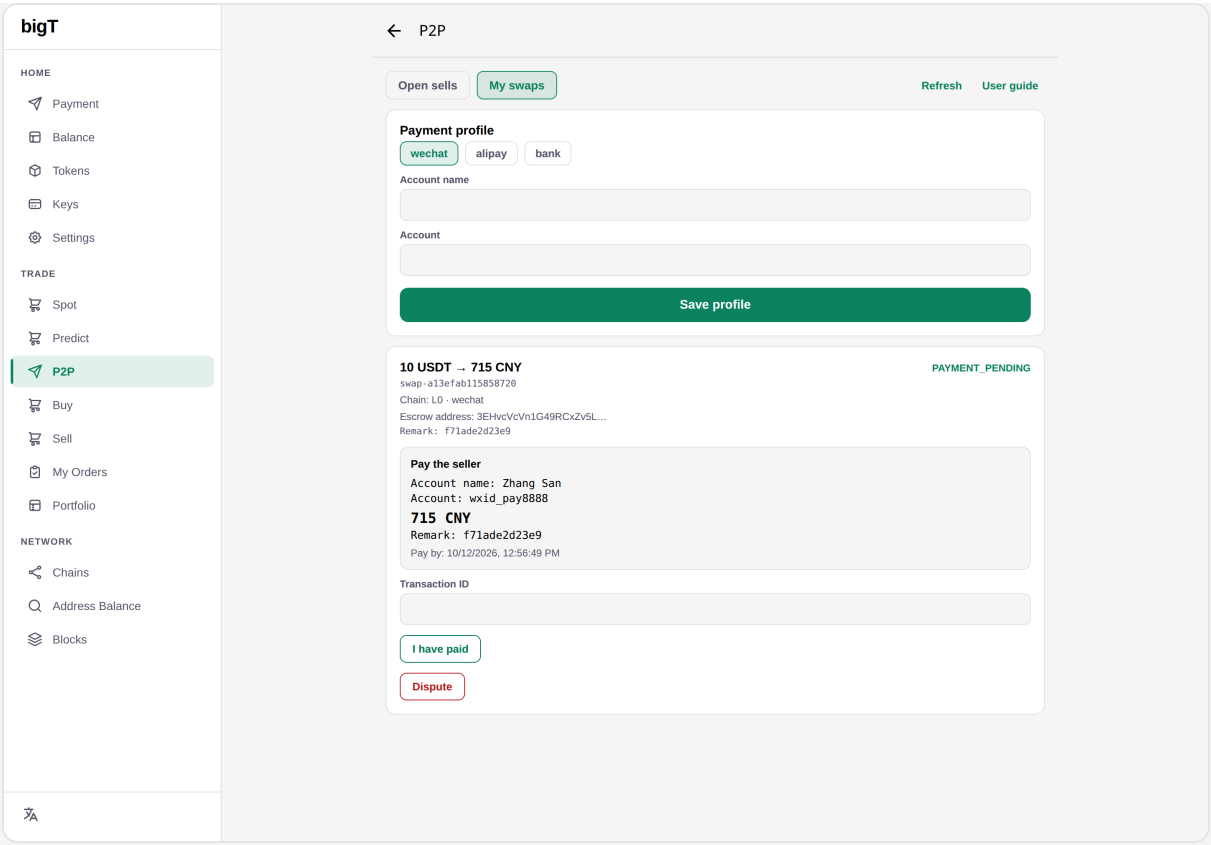
---

5. The buyer pulls the payment instructions

The buyer presses **Payment details**; the engine reveals the seller's profile

and mints a **per-swap remark code** the transfer *must* carry, plus the exact

amount and a deadline. The instructions render inline in the swap card:

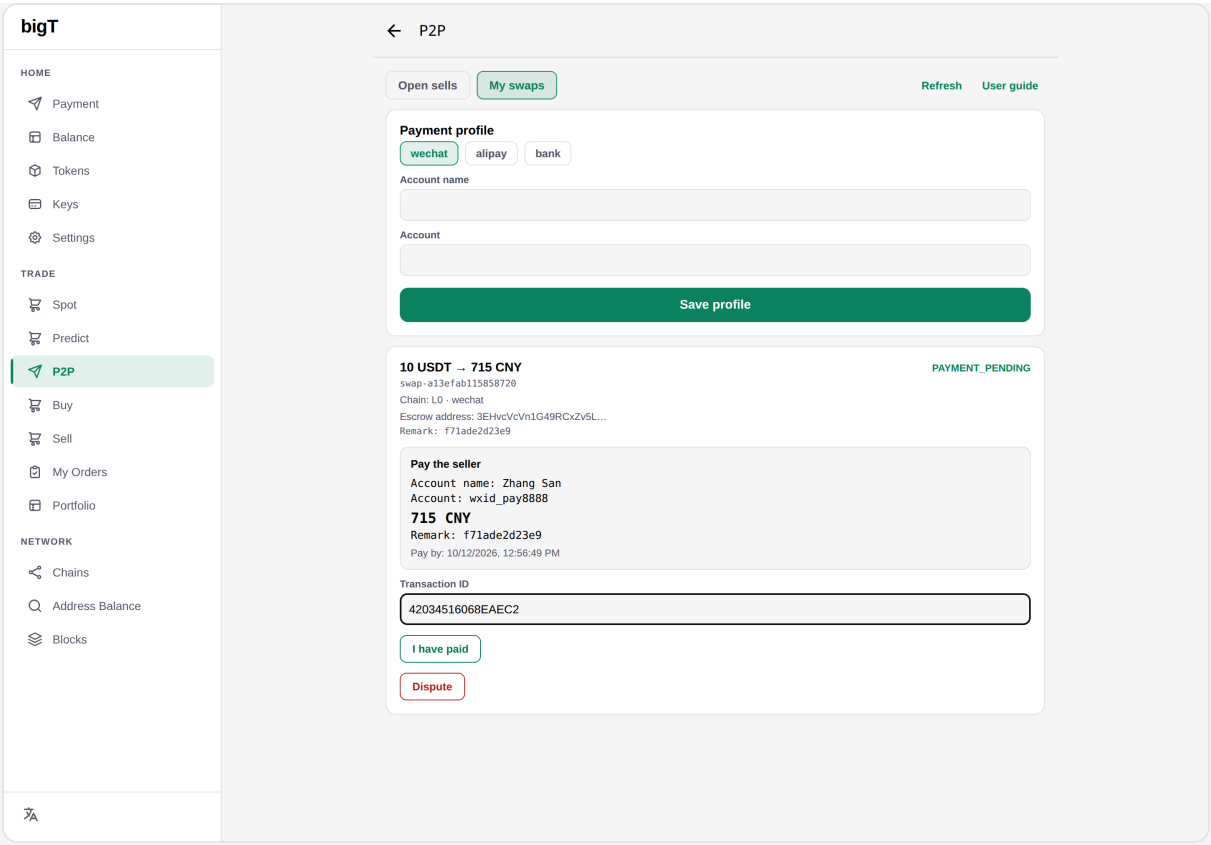


The swap is `PAYMENT_PENDING`, and a timeout starts — if the buyer never pays, the seller can expire and refund the escrow without the buyer's consent.

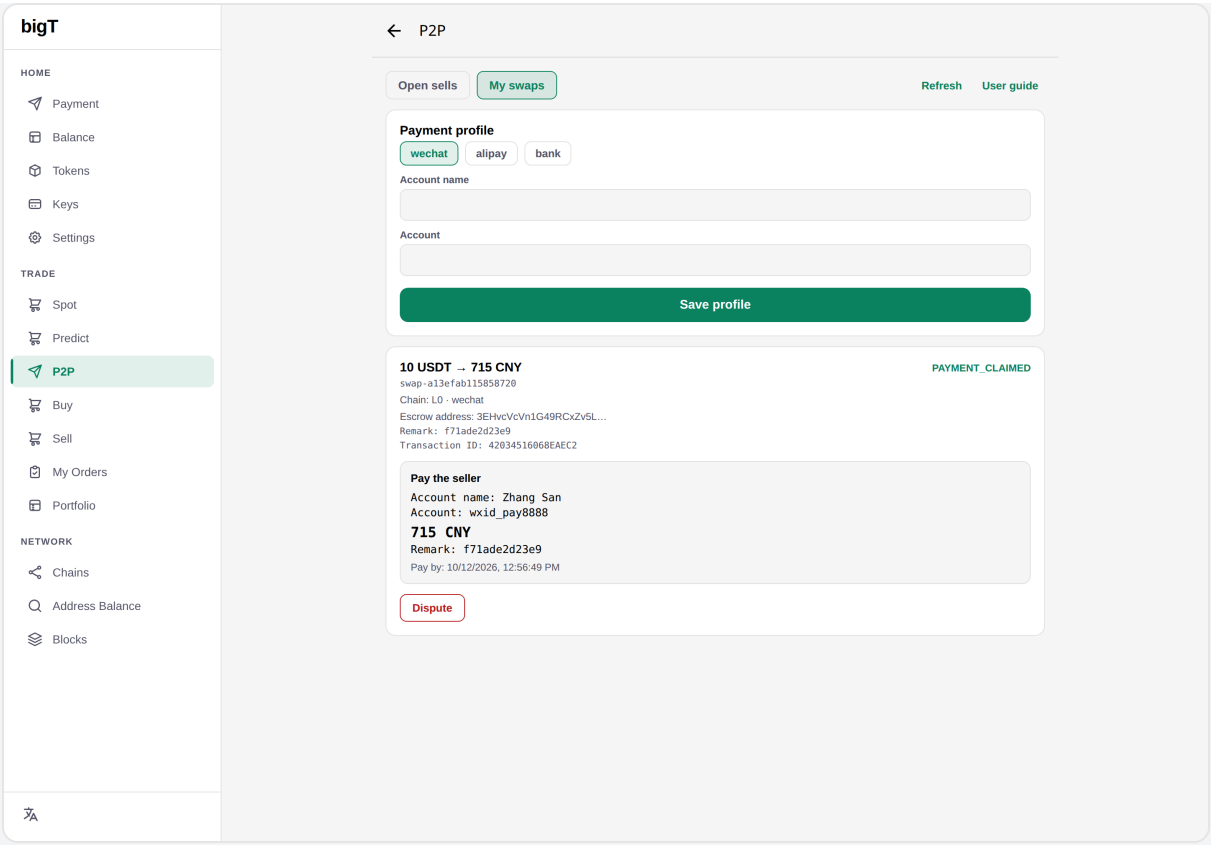
---

6. The buyer pays and claims with the transaction number

The buyer transfers the exact amount in their own WeChat/Alipay/bank app, pastes the transaction number into **Transaction ID**, and presses **I have paid**. An optional receipt image is stored server-side; only its `sha256` is anchored on the swap event:



Status is now PAYMENT_CLAIMED — **the tokens are still locked.** This state exists purely as the buyer's claim:



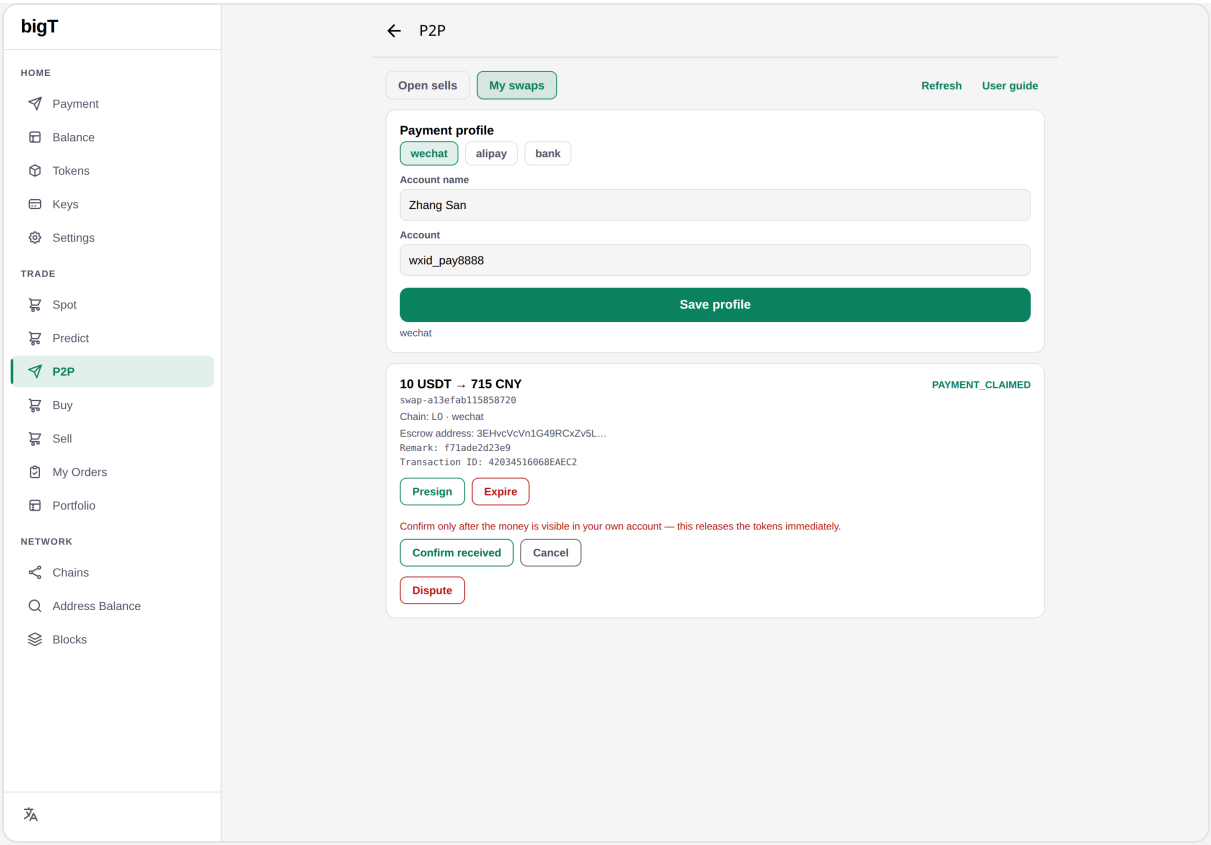
---

7. The seller confirms on their own statement

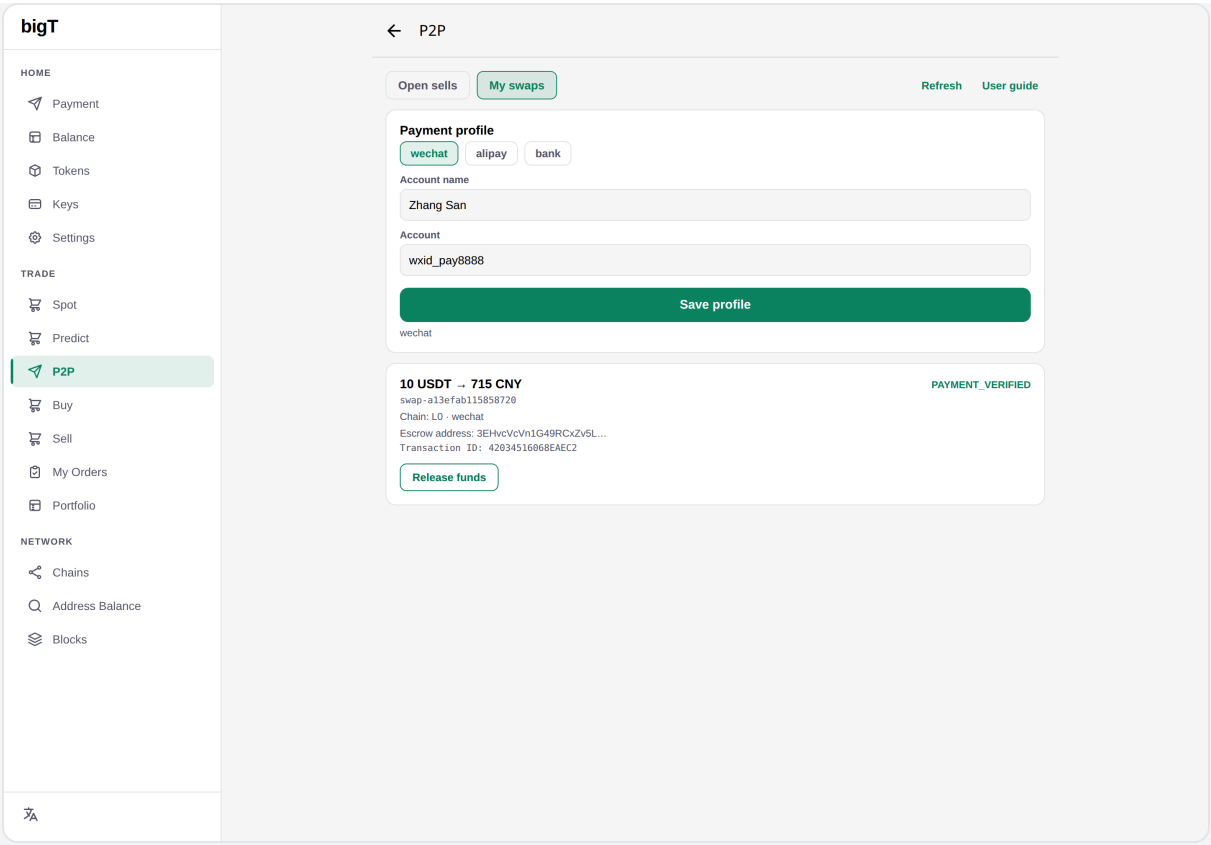
The seller's card shows the buyer's transaction number and the remark. The seller checks

amount + remark + time + transaction number in their own account and only then presses

Confirm received:



The swap is PAYMENT_VERIFIED :



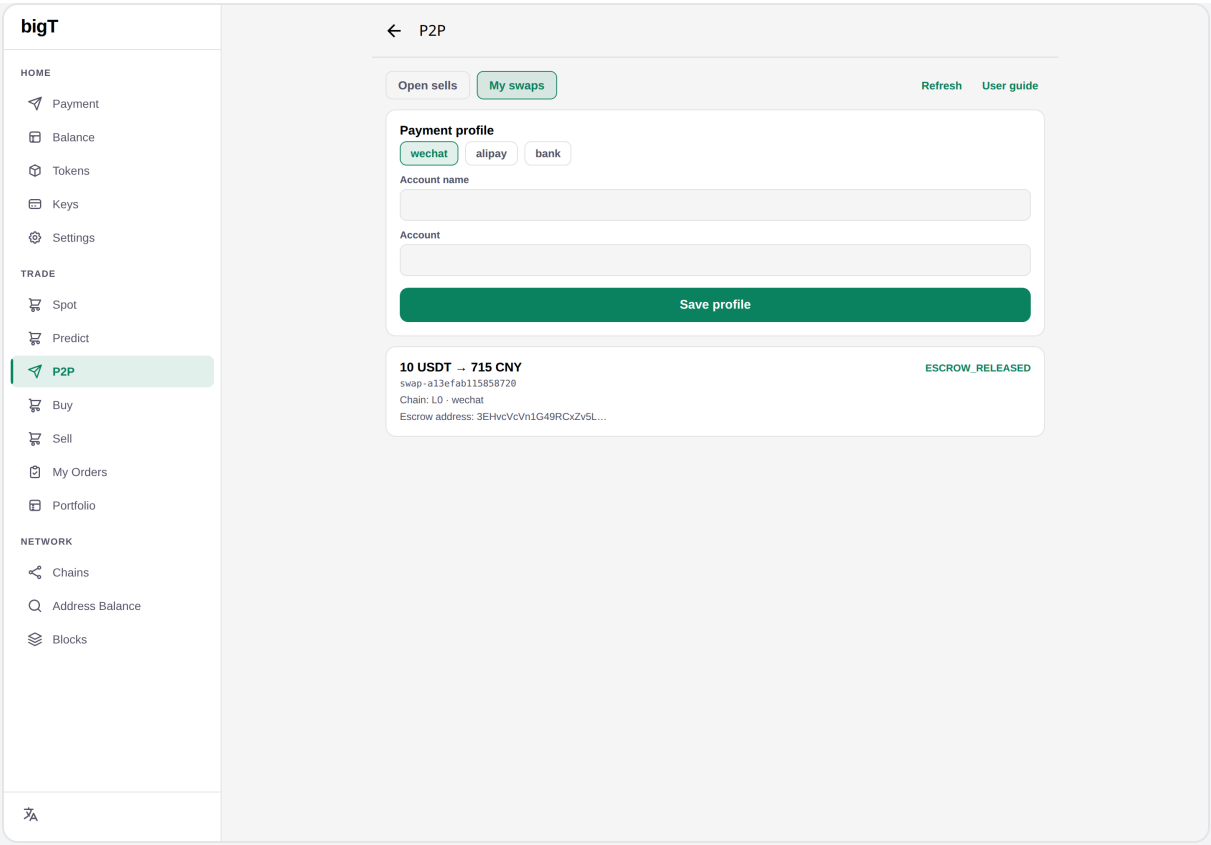
---

8. Release — the funds move

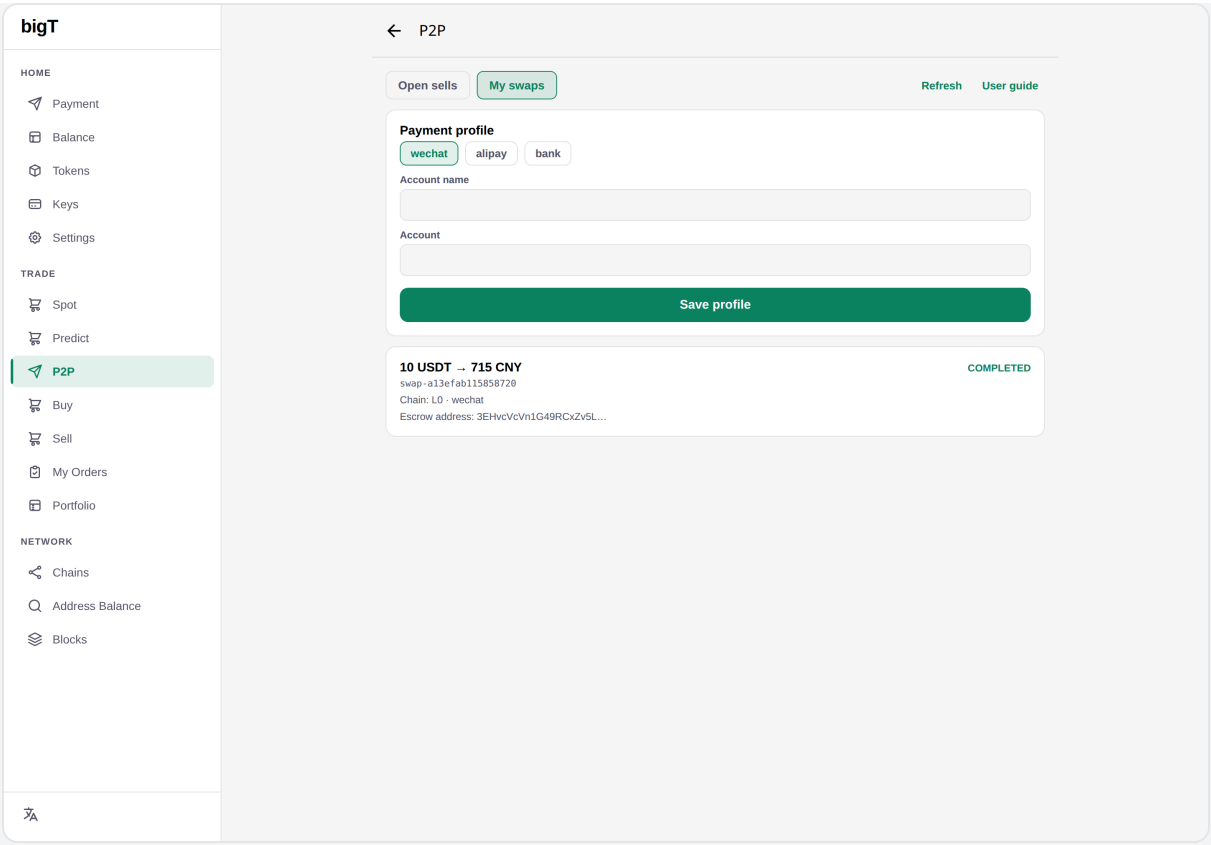
Release moves the escrowed tokens: the seller (presign or co-sign) and the

engine each sign the spend from the escrow output, and two signatures satisfy the

2-of-3 script. The tokens land at the buyer's receive address:



Because the seller was paid directly, there is no fiat payout step — the swap completes:



Optional — machine confirmation from your own phone

Step 7 is normally a manual tap. To have the engine confirm for you, pair your phone once and let it read the **seller's own** receipt — the same statement the seller would check by hand, read automatically.

Pair the phone (once). Install the **Mobile Agent** app (android-remote-control-mcp) on your own phone; it runs an MCP server *on the phone* (Accessibility service, no adb). Grant it Accessibility, start its server, and note the endpoint and the

short single-use code it shows. In the wallet open **Device**, press **Pair

with code**, and enter the endpoint and the code. Revoke the pairing any time

under **Settings › Access › Paired devices**.

Verify a swap. In the swap card press **Verify with my phone** — either

party may verify their own device. The engine connects to the endpoint you

paired and reads the matching entry from your own rail app (amount, remark, time

and transaction number), then the **wallet signs the verdict**. Alipay and bank

apps expose their screen to accessibility; WeChat hides its UI, so for WeChat

the app reads WeChat's own “payment received” notification:



What it changes. By default a read is only dispute evidence. Under the policy `SETTLEMENT_CNY_AUTO_RELEASE_ON` (`seller_receipt`, or the stricter `two_sided` / `verified_receipt`) a qualifying read **stands in for the manual confirm** and releases the escrow. It is still attestation, not an oracle: the pairing is the consent, it can be revoked at any time, and the engine reads only the one side you paired.

What protects you

Risk	Mitigation
Counterparty walks away	Funds sit in a 2-of-3 P2SH address; nobody can move them alone
Under- or over-payment	The amount is fixed at match; instructions carry the exact figure and a per-swap remark
Fake payment claim	The buyer's claim alone never releases — the seller confirms on their own statement
Seller claims non-receipt	Arbitration; the chain record and the proof store are the evidence
Engine goes rogue	Engine-only steps require the engine DID signature; every transition is anchored and publicly auditable
Dispute	Either party freezes the swap; the tokens stay locked until it resolves
Refund	`expire → refund`, co-signed by the engine and the seller — no buyer consent, no timelock

---

Going live

The demo runs against the in-repo settlement service with a memory store and no chain check. The real flow wires the escrow signer to the L0 broadcast step and runs the engine against L1-SOCIAL for the order book and the audit anchor; the buyer pays the seller directly, and the engine still only ever co-signs.